

IoT IDS Model Training Study Guide

Project: FYP_2026 (CNN-LSTM IDS + Federated Learning + CKKS HE + CVAE synthetic attack generator)

Purpose:

- Provide your teammate a single study document for how the model is trained, what epochs/rounds are run, which layers are used, and which metrics to inspect.

Core runtime files:

- fl_common/model.py (CNN-LSTM architecture)
- fl_client/client.py (client local training loop + epoch metrics)
- fl_server/server.py (round aggregation + checkpointing + history)
- fl_common/cvae.py (CVAE architecture used for synthetic attack windows)
- scripts/train_cvae.py (local CVAE training)
- scripts/preprocess_cicids2017.py (feature scaling + 10-step windows + client split)

Generated with scripts/generate_model_study_pdf.py

1) Data Pipeline and Feature Setup

Dataset and preprocessing:

- Source: CIC-IDS2017 CSV files (8 daily capture files)
- Preprocessing script: scripts/preprocess_cicids2017.py
- Features: fixed 78-feature order (EXPECTED_FEATURES)
- Labels for CNN-LSTM FL: binary (benign=0, attack=1)
- Scaling: StandardScaler saved as model/standard_scaler.pkl

Windowing for sequence model:

- Sequence length: 10 timesteps
- Window label = label of last row in each window
- Output chunks per client: X_seq_chunk_*.npy and y_seq_chunk_*.npy

Client partition default:

- bank_a: monday + tuesday
- bank_b: wednesday + thursday
- bank_c: friday

2) CNN-LSTM Architecture Used for FL Training

Model file: fl_common/model.py

Input/Output:

- Input tensor: (batch, seq_len=10, num_features=78)
- Output tensor: (batch, 1) raw logit (sigmoid applied for probability)

Layers:

1. Conv1d(78 -> 64, kernel_size=3, padding=1)
2. ReLU
3. LSTM(input_size=64, hidden_size=64, num_layers=1, batch_first=True)
4. Linear(64 -> 1)

Loss + threshold:

- Loss: BCEWithLogitsLoss with pos_weight=5.0
- Decision threshold: 0.5

Parameter count (from actual model object):

- Total trainable params: 48,385
- conv1.weight: 14,976
- conv1.bias: 64
- lstm.weight_ih_l0: 16,384
- lstm.weight_hh_l0: 16,384
- lstm.bias_ih_l0: 256
- lstm.bias_hh_l0: 256
- fc.weight: 64
- fc.bias: 1

HE-selected layers (CKKS aggregation target):

- lstm.weight_ih_l0
- lstm.weight_hh_l0
- fc.weight
- fc.bias

3) Epochs, Rounds, and Training Configuration

Where rounds/epochs are controlled:

- Backend API request model: backend/app/api/v1/fl.py (FLStartRequest)
- FL server env: backend/app/services/docker_service.py (start_fl_server)
- FL server runtime: fl_server/server.py
- FL client local training loop: fl_client/client.py (local_train)

Typical defaults used by the stack:

- num_rounds: API default 5 (server fallback default config 25 if env not supplied)
- local_epochs: 5
- learning_rate: 0.001
- batch_size: 128
- max_batches: 0 (means use all batches)
- min_clients: 1 (API default, can be increased)

Round lifecycle:

1. Server sends global weights and round config
2. Client trains locally for local_epochs
3. Client returns updated weights + metrics + optional signature
4. Server aggregates (FedAvg plain or FedAvg+HE)
5. Server checkpoints and logs round metrics

Security cadence in server:

- RECESS detection round every 5 rounds
- VSS proactive key refresh every 20 rounds

4) Metrics You Should Track (Python Graphs)

Current generated graph files (docs/assets):

- fl_aggregation_time.png
- fl_aggregation_type_distribution.png
- fl_round_coverage.png
- fl_metrics_summary.json

If history includes these keys, extra graphs are auto-generated:

- global_loss -> fl_global_loss.png
- global_accuracy -> fl_global_accuracy.png

How to regenerate graphs:

- python scripts/plot_training_metrics.py

Recommended study focus for teammate:

- Trend of global_loss and global_accuracy across rounds
- Aggregation time growth as number of clients/HE load increases
- Missing rounds/checkpoints as operational troubleshooting signals

Training History Snapshot:

- Entries in model/fl_training_history.json: 4
- Round range recorded: 1 to 4
- Average aggregation time: 0.0991 sec
- Aggregation types in history: {'fedavg_plain': 4}
- Global loss points available: 0
- Global accuracy points available: 0

Important note:

- Current history file mostly contains aggregation timing metadata. Loss/accuracy can still be captured from backend DB (fl_rounds) or by extending server history logging.

5) CVAE Training Model (Synthetic Attack Generator)

CVAE architecture file: `fl_common/cvae.py`

Training script: `scripts/train_cvae.py` (local) / `scripts/kaggle/train_cvae.py` (full Kaggle run)

CVAE purpose:

- Generate synthetic attack traffic windows (10 x 78) conditioned on class ID
- Used for realistic attack simulation during monitor/attack node runs

CVAE architecture:

Encoder: `Conv1d(78->64) + ReLU + LSTM(64->128) + FC(128->256) -> mu/log_var(128)`

Decoder: `FC((128 + 15)->256) + ReLU + BN + FC(256->512) + ReLU + BN + FC(512->780)`

Aux classifier: `Linear(128->15)` on latent mu

CVAE key outputs:

- `model/cvae_decoder.pt`
- `model/cvae_scaler.pkl`
- `model/cvae_class_centroids.pkl`

CVAE parameter count (from model object):

- Total: 785,243 | Encoder: 213,184 | Decoder: 570,124 | Aux: 1,935

6) How To Run End-to-End (Team Quick Start)

A) Setup

1. `./scripts/linux/setup.sh`
2. Login frontend with admin / admin123

B) Preprocess dataset (if needed)

- `python scripts/preprocess_cicids2017.py --clients 3 --window 10 --stride 1`

C) Start FL training from API (example)

- POST `/api/v1/fl/start` with JSON:

```
{"num_rounds": 25, "min_clients": 3, "use_he": true, "local_epochs": 5, "learning_rate": 0.001, "max_batches": 0}
```

D) Key outputs after training

- `model/global_final.pt`
- `model/fl_checkpoints/global_round_*.pt`
- `model/fl_training_history.json`

E) Plot metrics

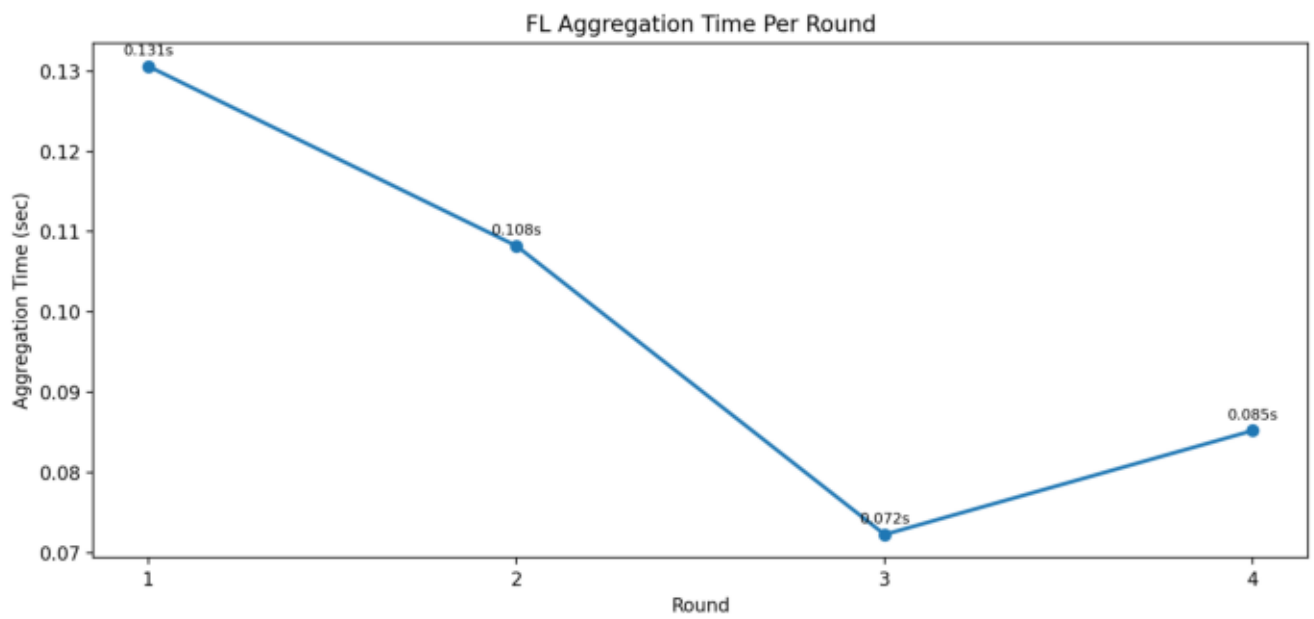
- `python scripts/plot_training_metrics.py`

F) Rebuild this PDF

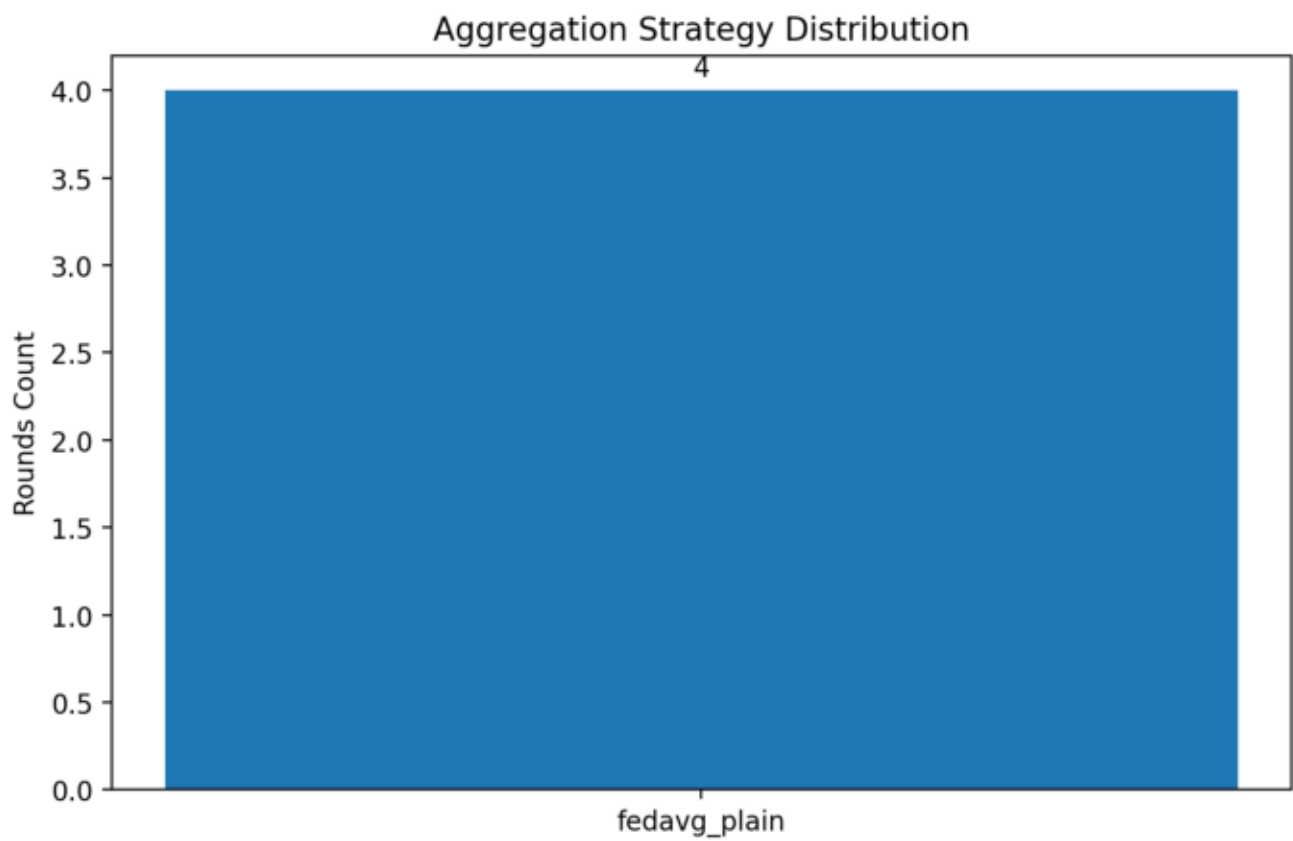
- `python scripts/generate_model_study_pdf.py`

Tip: Keep this PDF, the graph PNGs, and `fl_training_history.json` together for teammate review and presentation prep.

Appendix A - Aggregation Time Per Round

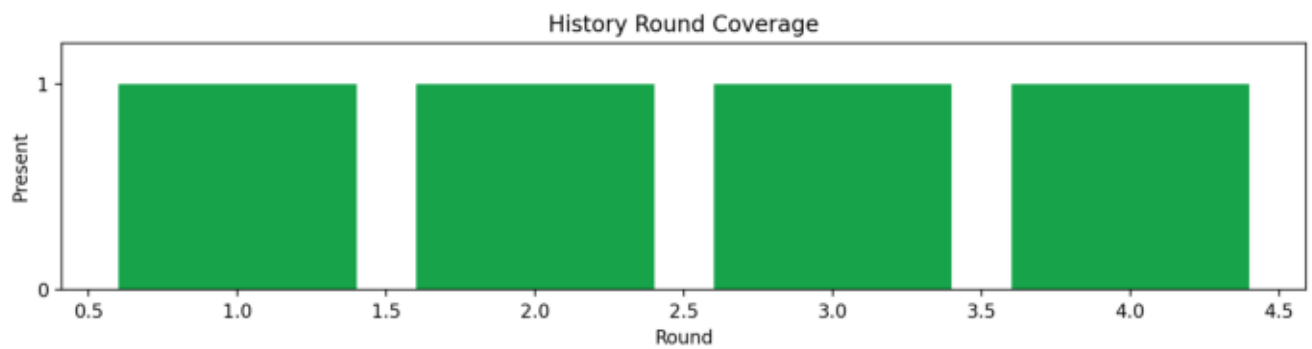


Appendix B - Aggregation Strategy Distribution



Shows how many history entries used each aggregation strategy.

Appendix C - Round Coverage



Green=round present in history, Red=missing within min..max round range.